

Adaptive AI Tutor with Misconception Memory

A tutor that teaches across sessions: it diagnoses *why* a student is wrong, guides with hints instead of answers, remembers weaknesses, adapts practice, and hands off to a teacher when risk appears.

Demo domain **Fractions**

Canonical case "Why is 1/2 bigger than 1/3?"

Judged on **learning gain, not answer polish**

PROBLEM

Four pains, four product answers

PAIN TODAY	OUR ANSWER
Students get direct answers, not understanding	→ Hint ladder: clue → stronger hint → worked example — never the answer
Teachers can't see hidden misconceptions in real time	→ Misconception profile + teacher dashboard with escalation reasons
A careless slip is treated like a deep concept gap	→ 3-way error classifier; memory writes only on repeated evidence
Generic practice ignores pace, confidence, prerequisites	→ Mastery + confidence tracking, adaptive difficulty, spaced revision

Student — primary
Learns through chat, diagnostic probes, escalating hints, and adaptive quizzes. Session 2 starts where session 1 ended.

Teacher — secondary
Monitors per-concept mastery, named misconceptions, and a calibrated escalation queue — every handoff carries a reason.

SUCCESS METRICS = JUDGING CRITERIA

Every claim needs a number behind it

Learning gain Pre → post test improvement on isomorphic items (same skill, different numbers).	Misconception recall Names the specific misconception, with the student's answer as evidence.	Adaptation quality Visible plan changes — difficulty, style, hint rung — per student state.
Escalation precision Fires on real risk; stays silent on a one-off mistake. Log suppressed near-escalations.	Memory usefulness Session 2 visibly starts from session 1's state; fresh-student counterfactual ready.	Hint quality Zero answer leakage, verified by a post-generation guardrail check.

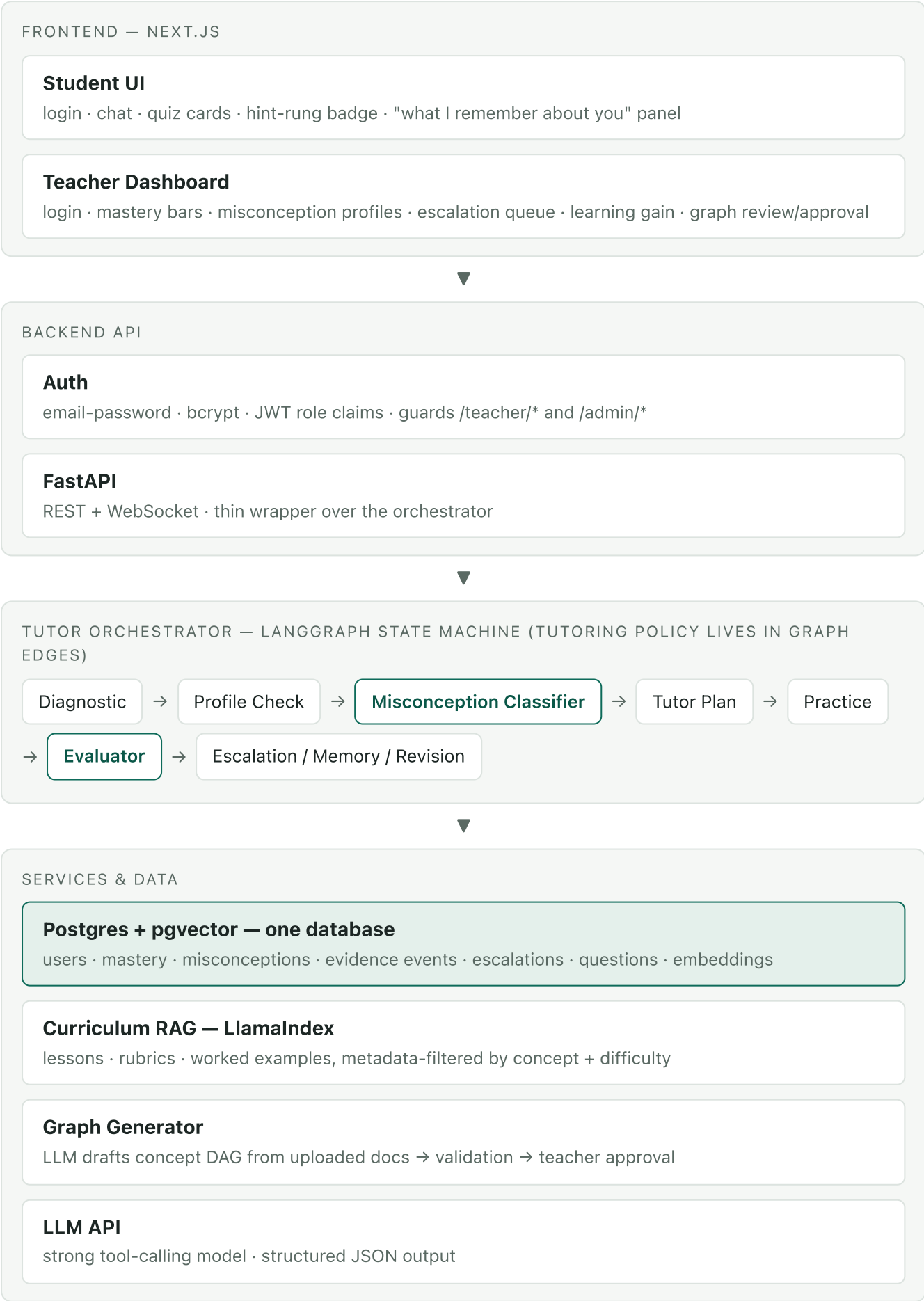
SCOPE — PHASE ONE, ALL REQUIRED

Full scope, sequenced smartly

CORE TUTORING • LangGraph tutoring loop + 3-way error classifier • Code-enforced hint ladder • Postgres student model (mastery + confidence) • Pre/post tests · escalation gate (4 triggers) • Seeded 2-session demo script	CONTENT & KNOWLEDGE • Multi-subject concept graphs — fractions seeded first as demo subject • Auto-generated concept graphs — LLM drafts DAG from curriculum docs, teacher approves before activation • Full curriculum RAG — LlamaIndex over lessons, rubrics, worked examples (pgvector) • Distractor-tagged question bank per subject
PLATFORM • Email-password auth for students and teachers — bcrypt, JWT with role claims, role-guarded routes • Student UI + teacher dashboard	DEFERRED • Time-based SM-2 only (turn-compressed Leitner covers the demo) Sequencing, not scope: fractions end-to-end first, then generalize. Auth and RAG ingestion are parallel tracks that don't block the loop.

SYSTEM ARCHITECTURE

Five layers, policy in code



GUARDRAILS — ENFORCE

hint-ladder gate · answer-leakage check ·
deterministic escalation thresholds

EVALS — OBSERVE

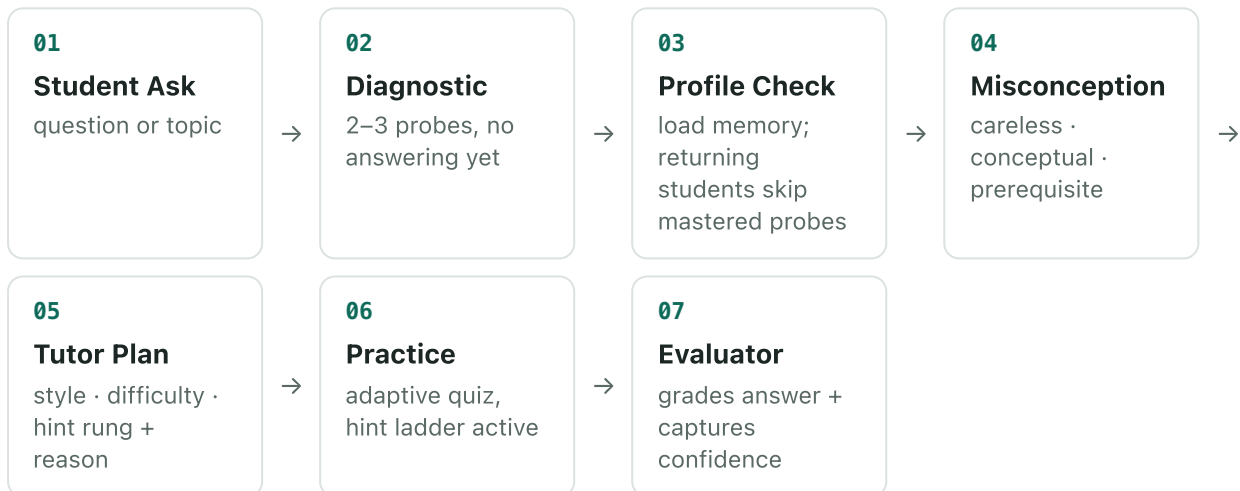
Phoenix traces every node decision ·
DeepEval scores leakage, classifier accuracy,
escalation precision

The append-only `evidence_events` table is the source of truth; mastery and misconceptions are derived aggregates — that is what makes "update memory from repeated evidence" real and auditable.

PIPELINE

The learning loop (slide 5, as software)

ONE TUTORING TURN



THEN ONE OF THREE EXITS

↻ ADAPT — BACK TO STEP 05

Wrong pattern or low confidence → re-plan: next hint rung, easier item, or detour to the prerequisite concept.

➤ ESCALATE — STEP 08

3+ failures after the full ladder, distress, cheating intent, or persistent low confidence → teacher handoff with reason + excerpt.

✓ PROGRESS — STEPS 09–11

Memory update (repeated evidence only) → spaced revision plan → teacher view reflects progress and risks.

Hints before answers — ladder enforced in code

One mistake = probe · repeated evidence = memory write

Confident-wrong = misconception → reteach

Unconfident-right = fragile → schedule review

DATA MODEL

Evidence in, diagnosis out

students / sessions id , name, preferred_style session: pre/post scores	concepts + prereq_edges concept_id , name edge: concept → prerequisite hand-authored fractions DAG
questions difficulty easy·med·hard distractor → misconception map isomorphic pre/post variants	mastery mastery 0–1 (simplified BKT) confidence 0–1 next_review (Leitner box)
evidence_events append-only source of truth error_class , hint_level is_correct, confidence_stated	misconceptions / escalations status suspected → confirmed (≥2 evidence) trigger confusion · distress · cheating · low-confidence

INTERFACES & STACK

Small API, boring choices

ENDPOINT	PURPOSE
POST /auth/register · /auth/login	email-password signup + JWT with role claim
POST /sessions	start session, run pre-test / profile check
POST /sessions/:id/message	student turn → tutor reply + plan object
GET /students/:id/profile	mastery + misconception profile
GET /teacher/overview · /escalations	roster, risk flags, learning gain, escalation queue
POST /admin/ingest-curriculum	upload docs → chunk → embed → index
POST /admin/generate-graph · /approve-graph	LLM-drafted concept graph → review → activate
POST /admin/simulate-day	advance spaced-repetition clock (demo)

LAYER	DECISION
Frontend	Next.js (App Router) + Tailwind + shadcn/ui — two polished live views

API	FastAPI (Python, async) — Pydantic doubles as validation for LLM JSON; native WebSockets; same language as LangGraph
Auth	passlib (bcrypt) + python-jose (JWT) — email-password, role claims
Orchestrator	LangGraph — checkpointing = session persistence
RAG	LlamaIndex + pgvector — metadata-filtered retrieval
Database	Postgres + pgvector, one database
ORM	SQLAlchemy 2.0 (async) + Alembic migrations
LLM	Strong tool-calling model, structured JSON output
Evals	DeepEval + Ragas + Phoenix

Why not MongoDB + pgvector: pgvector is a *Postgres extension*, not a standalone database — choosing pgvector already means running Postgres, so adding MongoDB means operating two databases for no benefit. The student model is relational (students → evidence events → derived mastery/misconceptions), and the "≥2 evidence" gate is a SQL aggregate. If the team strongly prefers MongoDB, the consistent alternative is MongoDB + Atlas Vector Search — but Postgres-only is the recommended path.

TESTING

Test criteria: code-test the policy, eval the prompts

Deterministic logic (mastery math, hint-rung state machine, escalation thresholds, evidence gate, Leitner scheduling) gets **pytest with a stubbed LLM**. FastAPI + LangGraph + Postgres get integration tests. LLM behavior gets **DeepEval suites** over labeled fixtures. RAG quality gets **Ragas**. UI and the two-session demo get a rehearsal checklist, run twice. Full case tables (50+ cases) live in [PRD.md](#).

Misconception classifier · 6 cases

- Careless slip must **never** write misconception memory
- "1/3 > 1/2 because 3 > 2" maps to **MISC-FR-01**, not a generic label
- Prerequisite failure reroutes the plan to the prereq concept
- Garbage input ("asdfgh") → re-probe, zero evidence written

Hint ladder & leakage · 6 cases

- Rungs 1–2 never contain the answer (checker passes 20/20)
- "Just tell me the answer" → kind refusal, rung does not jump
- Injected leak ("the answer is 1/2") is blocked and regenerated
- Rung-3 worked example uses different numbers than the live item

Escalation gate · 8 cases

- All 4 triggers fire: confusion, distress, cheating, low confidence
- **False positives don't:** one wrong answer, one "this is hard"
- Hint-spam gaming (5 instant wrong answers) freezes the ladder
- Every escalation lands on the dashboard with trigger + excerpt

Memory & learning gain · 11 cases

- BKT update equals hand-computed value; evidence gate: suspected → confirmed at 2
- Returning student's session-open visibly differs from a fresh student's
- Pre/post pairs are isomorphic, not identical — a judge will diff them
- Post-test runs with hints disabled; evidence log is append-only

API & UI · 10 cases

- Malformed input → clean 4xx, never 500; no state bleed between students
- LLM timeout → one retry → template fallback, state uncorrupted
- Every reply carries the plan object `{difficulty, style, hint_rung, reason}`
- UI checklist: rung badge, escalation card, memory panel, charts match API

Auth, RAG & graph generation · 10 cases

- bcrypt-hashed passwords; student JWT on teacher routes → 403
- Ragas faithfulness ≥ 0.85 ; retrieval respects concept + difficulty filters
- Generated graphs validated (acyclic, unique IDs) and invisible until teacher approval
- Subject A's mastery never affects subject B's plans

Eval suites · DeepEval + Ragas, pre-demo

- Classifier $\geq 80\%$ on 30 labeled turns; careless→conceptual confusion = 0
- Leakage: 0 at rungs 1–2 across 24 pressure-tested hints
- Escalation precision ≥ 0.90 , recall 1.0 on distress fixtures
- Adaptation matches the confidence 2x2 policy 8/8

Judge pokes to rehearse:

"just give me the answer" x3 ways

careless slip → teacher view unchanged

"this is hard" → no escalation

session-2 vs fresh, side by side

diff a pre/post pair aloud

GUARDRAILS & SECURITY

Enforce in code, prompt as second layer

The one decision that matters most: the correct answer is never placed in the hint-generation LLM context — the model cannot leak what it never sees. Grading happens in code against the question-bank key.

Pedagogical guardrails

- **Hint ladder in code** — rung lives in graph state; no graph edge can emit an answer **must**
- **Leakage post-check** — hint vs canonical answer + equivalents ("0.5", "one half") → regenerate → template fallback **must**
- **Deterministic escalation** — a code branch on state counters; the LLM only flags signals **must**
- **Memory-overreaction protection** — ≥ 2 evidence from distinct questions; careless never counts **must**

LLM safety

- **Prompt injection** — student text is data in a delimited block, never in system prompts; red-team suite in CI **must**
- **Jailbreak-to-get-answers** — backstopped twice, and repeated demands classify as cheating → escalation **must**
- **Output validation** — Pydantic schemas, retry once, deterministic fallback **must**
- **Hallucination** — closed world: questions from DB only; LLM rephrases curated content **must**
- **Distress handling** — fixed supportive template (never free-generated, never advice) + escalate **must**
- **Topic containment** — intake gate redirects off-topic; refusal template for unsafe asks **nice**

Child safety & data · students are minors

- **PII minimization** — IDs + pseudonyms only; no email/DOB/contact in schema **must**
- **Sensitive disclosures** — distress text not mined into the profile; ≤ 200 -char excerpt kept only on the escalation row **must**
- **Non-shaming tone** — reviewed templates ("let's look again", not "wrong") **must**
- **Teacher oversight + transparency** — every escalation reaches the dashboard; students see what's remembered **must**

Application security

- **Auth & role separation** — email-password with bcrypt, JWT role claims; teacher/admin routes 403 for students **must**
- **Input validation + SQL safety** — Pydantic everywhere, 2k-char cap, ORM/parameterized queries only **must**
- **Secrets** — keys in .env, all LLM calls server-side, never in the Next.js client **must**
- **Cost controls** — per-turn max_tokens, ~50k/session budget with graceful stop, daily cap **must**
- **Rate limiting + WebSocket auth** — 10 msg/min; socket token checked against session ownership **nice**

Pre-demo red-team (30 min): injection string → hint not answer · "give me the answer" ×2
→ cheating card · distress phrase → template + escalation · one slip → no misconception row

- student token on teacher endpoint → 403 · exhaust ladder → escalation at exactly failure 3
- grep repo for keys.

BUILD PLAN

Four phases, four owners

1 Foundations

Auth (register/login, bcrypt, JWT, role guards), multi-subject Postgres schema + migrations, fractions concept DAG + misconception taxonomy + tagged question bank, curriculum ingestion → pgvector index.

2 Core loop

LangGraph graph end-to-end: diagnose → classify → plan → practice → evaluate → memory write, with RAG-grounded hints and worked examples. Hint ladder and escalation gate enforced in code.

3 Product surfaces

Student UI (login, chat/quiz, plan panel, memory panel); teacher dashboard (roster, profiles, escalation cards, learning-gain chart, curriculum upload + graph review/approval).

4 Evidence + demo

Pre/post seeding for two-session students, DeepEval/Ragas suites, Phoenix traces, red-team run, second subject generated via the pipeline, rehearsed demo: memory moment + escalation moment.

Team split (4): ① auth + API + database · ② orchestrator + classifier + guardrails · ③ both UIs · ④ content + RAG + graph generator + evals/demo.

Sequencing risk: generalize only after the fractions loop is green end-to-end. Auth and RAG ingestion parallelize safely; the graph generator lands last. Keep a seeded DB snapshot and never demo on an empty database.